



浙江大學
ZHEJIANG UNIVERSITY

计算机组成与设计实验项目报告

cache 仿真器与性能分析

姓名 _____

学号 _____

院所 _____ 信息与工程学院

2024 年 12 月 20 日

目录

1	项目介绍	3
2	项目任务与要求	3
3	任务 1:trace-driven cache simulator 具体实现	5
3.1	cache 初始化	5
3.2	基础功能的 cache 实现	5
3.3	增加可变关联性功能	6
3.4	增加数据写和指令读的统计	7
3.5	增加 split cache 功能	10
3.6	增加 write through 和 write no allocate 策略	10
4	任务 2: 评估不同的高速缓存体系结构特性的性能	15
4.1	spice.trace	19
4.2	cc.trace	19
4.3	tex.trace	19
5	项目总结	26

1 项目介绍

实现一个跟踪驱动的缓存模拟器，并使用它来评估不同的缓存体系结构特性的性能。

2 项目任务与要求

任务 1: 实现 trace-driven cache simulator

高速缓存模拟器将可以根据命令行中给出的参数进行配置，并且必须支持以下功能：

- Total cache size
- Block size
- Unified vs. split I- and D-caches
- Associativity
- Write back vs. write through
- Write allocate vs. write no allocate

模拟器必须跟踪：

- Number of instruction references
- Number of data references
- Number of instruction misses
- Number of data misses
- Number of words fetched from memory
- Number of words copied back to memory

任务 2: 评估不同的高速缓存体系结构特性的性能

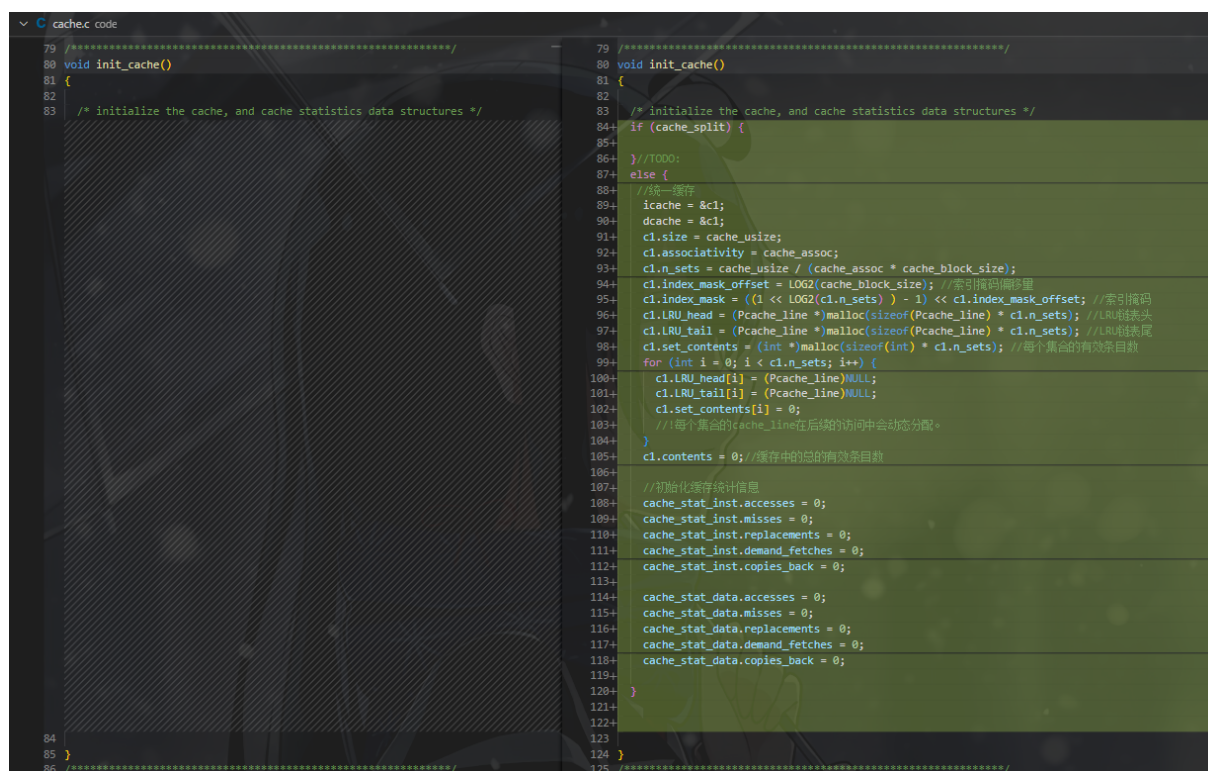
- characterize the working set size of the three sample traces given.
- evaluate the Impact of Block Size on performance
- evaluate the Impact of Associativity on performance
- evaluate the Impact of Memory Bandwidth on performance

3 任务 1:trace-driven cache simulator 具体实现

3.1 cache 初始化

先实现 cache 的初始化,命令行参数获取和设置代码框架已经给出,只需要在 cache.c 中实现 init_cache 函数即可。

具体代码实现如下图所示,这里先考虑最简单的情况,即 i-d 统一,直接映射,写回,写分配的 cache。该函数的功能是初始化 cache,根据 cache.c 中已经设置好的静态变量,初始化 c1 结构体中的各个参数并分配内存。同时初始化统计信息的结构体 cache_stat_inst 和 cache_stat_data。



```

79 //*****
80 void init_cache()
81 {
82     /* initialize the cache, and cache statistics data structures */
83
84     if (cache_split) {
85         //TODO:
86     }
87     else {
88         //统一缓存
89         icache = &c1;
90         dcache = &c1;
91         c1.size = cache_size;
92         c1.associativity = cache_assoc;
93         c1.n_sets = cache_size / (cache_assoc * cache_block_size);
94         c1.index_mask_offset = LOG2(cache_block_size); //索引掩码偏移量
95         c1.index_mask = ((1 << LOG2(c1.n_sets)) - 1) << c1.index_mask_offset; //索引掩码
96         c1.LRU_head = (Pcache_line *)malloc(sizeof(Pcache_line) * c1.n_sets); //LRU链表头
97         c1.LRU_tail = (Pcache_line *)malloc(sizeof(Pcache_line) * c1.n_sets); //LRU链表尾
98         c1.set_contents = (int *)malloc(sizeof(int) * c1.n_sets); //每个集合的有效条目数
99         for (int i = 0; i < c1.n_sets; i++) {
100             c1.LRU_head[i] = (Pcache_line)NULL;
101             c1.LRU_tail[i] = (Pcache_line)NULL;
102             c1.set_contents[i] = 0;
103             //每个集合的cache_line在后续的访问中会动态分配。
104         }
105         c1.contents = 0; //缓存中的总的有效条目数
106
107         //初始化缓存统计信息
108         cache_stat_inst.accesses = 0;
109         cache_stat_inst.misses = 0;
110         cache_stat_inst.replacements = 0;
111         cache_stat_inst.demand_fetches = 0;
112         cache_stat_inst.copies_back = 0;
113
114         cache_stat_data.accesses = 0;
115         cache_stat_data.misses = 0;
116         cache_stat_data.replacements = 0;
117         cache_stat_data.demand_fetches = 0;
118         cache_stat_data.copies_back = 0;
119     }
120 }
121
122
123
124 }
125 //*****

```

图 1: cache 初始化

3.2 基础功能的 cache 实现

接下来实现 perform_access 函数,该函数对 cache 进行仿真,根据访问的地址和类型,更新 cache 的状态。

具体代码实现如下图所示:

同样先实现 unify cache 的情况,

先根据 address 计算出 tag 和 index,然后根据 index 找到对应的 cache 组,如果缓存组为空,说明没有命中,需要动态分配一个 cache 块,同时更新统计信息。

否则缓存组不为空,根据双向链表的数据结构,遍历链表,找到对应的 cache 块,(目前 associativity 为 1,即直接映射代码尚未实现)。如果命中,则更新 cache 块的状态。

态，同时更新统计信息。如果未命中，则根据写策略和写分配策略，更新 cache 块的状态，同时更新统计信息。

```
88 void perform_access(addr, access_type)
89 {
90     unsigned addr, access_type;
91 }
92
93 /* handle an access to the cache */
127
128 void perform_access(addr, access_type)
129 {
130     unsigned addr, access_type;
131 }
132
133 /* handle an access to the cache */
134
135 //开始处理缓存访问
136 int index, tag;
137 //读入的地址为
138 if (cache_split) {
139     //TODO:
140 }
141 else {
142     //统一缓存
143     index = (addr & c1.index_mask) >> c1.index_mask_offset;
144     tag = addr >> (c1.index_mask_offset + LOG2(c1.n_sets));
145     //在LRU链表中查找缓存行
146
147     //先考虑associativity为1的情况
148     //先只考虑read, 不考虑write
149     Pcache_line curr_cache_set_head = c1.LRU_head[index]; //当前缓存组
150     if (curr_cache_set_head == NULL) {
151         // 初始状态, 分配一个初始的缓存行
152         Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
153         temp_cache_line->tag = tag; //设置缓存行的tag
154         temp_cache_line->LRU_next = NULL; //设置缓存行的下一个指针
155         temp_cache_line->LRU_prev = NULL; //设置缓存行的上一个指针
156         temp_cache_line->dirty = FALSE; //设置缓存行的脏位
157         insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); //将缓存行插入到LRU链表的头部
158     }
159     //更新统计信息, 先使用data设计信息, 这里还未添加split cache的情况
160     cache_stat_data.accesses++;
161     cache_stat_data.misses++;
162     cache_stat_data.demand_fetches += words_per_block;
163 }
164 else {
165     //应该遍历当前缓存组, 先考虑associativity为增的情况
166     if (curr_cache_set_head->tag == tag) {
167         // 命中
168         cache_stat_data.accesses++;
169     }
170     else {
171         // 未命中
172         cache_stat_data.accesses++;
173         cache_stat_data.misses++;
174         cache_stat_data.demand_fetches += words_per_block;
175         // 替换
176         Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
177         temp_cache_line->tag = tag; //设置缓存行的tag
178         temp_cache_line->LRU_next = NULL; //设置缓存行的下一个指针
179         temp_cache_line->LRU_prev = NULL; //设置缓存行的上一个指针
180         temp_cache_line->dirty = FALSE; //设置缓存行的脏位
181         delete(&c1.LRU_head[index], &c1.LRU_tail[index], curr_cache_set_head); //删除当前缓存组的头部
182         insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); //将新的缓存行插入到LRU链表的头部
183         cache_stat_data.replacements++;
184     }
185 }
186 }
```

图 2: cache 实现

3.3 增加可变关联性功能

接下来实现可变关联性的 cache，即 set-associative cache。

如下图 3 所示：首先在初始分配 cache 组的时候，需要加上 cache 结构体中对每组的 cache 块的有效条目数和总的条目数的设置。

如下图 4 所示：增加遍历 cache 组的功能，寻找对应的 cache 块。然后判断是否命中，更新 cache 块的状态和统计信息。如果未命中，还要判断组是否已满，如果未满，则动态分配一个 cache 块，否则根据 LRU 替换策略，选择一个 cache 块替换。然后更新 cache 的统计信息。

这里的 LRU 替换就是将最近使用的 cache 块放在链表的头部，最久未使用的 cache 块放在链表的尾部，每次替换时，将链表尾部的 cache 块替换掉。

```

127 void perform_access(addr, access_type)
128 unsigned addr, access_type;
129 {
130     /* handle an access to the cache */
131     //开始处理缓存访问
132     int index, tag;
133     //获取index和tag
134     if (cache_split) {
135         //cache_split
136     }
137     else {
138         //统一操作
139     }
140     index = (addr & ci_index_mask) >> ci_index_mask_offset;
141     tag = addr >> (ci_index_mask_offset + LOG2(ci_n_sets));
142     //在index表中查找缓存行
143     //向cache_line数组中的index[i]写入的情况
144     //向只读集合read，不向write
145     Patch_line curr_cache_set_head = ci_LRU_head[index];
146     if (curr_cache_set_head == NULL) { //当前缓存为空
147         temp_cache_line = LRU_prev = NULL; //当前缓存行上的上一个指针
148         temp_cache_line_index = NULL; //当前缓存行上的index
149         insert(ci_LRU_head[index], ci_LRU_tail[index], temp_cache_line); //将缓存行插入到当前缓存表的头部
150     }
151     //更新统计信息，更新命中统计值，这里还未涉及split cache的情况
152     cache_stat_data.accesses++;
153     cache_stat_data.misses++;
154     cache_stat_data.demand_fetches += words_per_block;
155     else {
156         //在更新统计信息之前，先向cache_line数组中的index[i]写入的情况
157     }
158     temp_cache_line = LRU_prev = NULL; //当前缓存行上的上一个指针
159     temp_cache_line_index = NULL; //当前缓存行上的index
160     insert(ci_LRU_head[index], ci_LRU_tail[index], temp_cache_line); //将缓存行插入到当前缓存表的头部
161     ci_cache_contents[index]++; //当前缓存命中次数增加
162     ci_cache_contents[index] = ci_cache_contents[index] >> LOG2(words_per_block);
163     //更新统计信息，更新命中统计值，这里还未涉及split cache的情况
164     cache_stat_data.accesses++;
165     cache_stat_data.misses++;
166     cache_stat_data.demand_fetches += words_per_block;
167     else {
168         //在更新统计信息之前，先向cache_line数组中的index[i]写入的情况
169     }

```

图 3: 可变关联性 cache

```

162     else {
163         // 设置临时关联缓存, 为后续associativity为缓存设置
164
165         if (curr_cache_set_head == tag) {
166             // 命中
167             cache_stat_data.accesses++;
168         }
169         else {
170             // 未命中
171             cache_stat_data.accesses++;
172             cache_stat_data.misses++;
173             cache_stat_data.demand_fetches += words_per_block;
174
175             // 替换
176             Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
177             temp_cache_line->tag = tag; //设置缓存行tag
178             temp_cache_line->LRU_next = NULL; //设置缓存行下一个指针
179             temp_cache_line->LRU_prev = NULL; //设置缓存行的上一个指针
180             temp_cache_line->dirty = FALSE; //设置缓存行脏位
181
182             delete(&LRU_head_index), &LRU_tail_index, curr_cache_set_head; //删除当前缓存行指针变量
183
184             insert(&LRU_head_index, &LRU_tail_index, temp_cache_line); //将新的缓存行插入到LRU链表头部
185             cache_stat_data.replacements++;
186         }
187     }
188 }
189
190 else {
191     //临时关联缓存命中, 当前缓存行
192     Pcache_line curr_cache_set_tail = &LRU_tail_index; //当前缓存行尾部
193     Pcache_line curr_cache_line = &LRU_head_index; //当前缓存行头部
194     //开始遍历
195     while (curr_cache_line != NULL) {
196         if (curr_cache_line->tag == tag) {
197             // 命中
198             break;
199         }
200         else {
201             curr_cache_line = curr_cache_line->LRU_next; // tag == word_id 为变量 associativity 关联变量
202         }
203     }
204     if (curr_cache_line != NULL) {
205         // 命中
206         cache_stat_data.accesses++;
207         cache_stat_data.misses++;
208         cache_stat_data.demand_fetches += words_per_block;
209         cache_stat_data.replacements++;
210
211         // 替换
212         Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
213         temp_cache_line->tag = tag; //设置缓存行tag
214         temp_cache_line->LRU_next = NULL; //设置缓存行下一个指针
215         temp_cache_line->LRU_prev = NULL; //设置缓存行的上一个指针
216         temp_cache_line->dirty = FALSE; //设置缓存行脏位
217         if (curr_cache_line->dirty == false) {
218             // 命中, 当前缓存行脏位为false
219             delete(&LRU_head_index), &LRU_tail_index; //删除当前缓存行指针变量
220             insert(&LRU_head_index, &LRU_tail_index, temp_cache_line); //将新的缓存行插入到LRU链表头部
221         }
222         else {
223             // 命中缓存, 当前缓存行脏位为true
224             insert(&LRU_head_index), &LRU_tail_index, temp_cache_line); //将新的缓存行插入到LRU链表尾部
225             delete(&curr_cache_line->dirty); //删除缓存行脏位变量
226             curr_cache_line->dirty = false; //设置缓存行脏位为false
227         }
228     }
229 }
230 }
231 }

```

图 4: 可变关联性 cache

3.4 增加数据写和指令读的统计

目前的 cache 只能实现对 data 的读取，接下来需要增加对指令的读取和数据写的统计。(目前默认按照 write back 和 write allocate 的策略先进行实现)

如下图 5 所示：首先在分配 cache 块的时候，需要加上对 cache 块的 dirty 标志位的设置。判断是否是数据写，如果是数据写，需要将 dirty 标志位置为 1。在更新统计信息的时候，需要根据访问的类型，更新对应的统计信息。


```

127 //*****
128 void perform_access(addr, access_type)
129 {
130     unsigned addr, access_type;
131 }
132 /* handle an access to the cache */
133 //开始处理缓存访问
134 int index, tag;
135 //存入的地址为
136 if (cache_split) {
137     //1000:
138 }
139 else {
140     //统一缓存
141     //计算索引和标签
142     index = (addr & c1.index_mask) >> c1.index_mask_offset;
143     tag = addr >> (c1.index_mask_offset + LOG2(c1.n_sets));
144     //在关联表中查找缓存行
145     //先考虑read, 不考虑write
146     Pcache_line curr_cache_set_head = c1.LRU_head[index]; //当前缓存组
147     if (curr_cache_set_head == NULL) {
148         // 初始状态, 分配一个新的缓存行
149         Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
150         temp_cache_line->tag = tag; //设置缓存行的内容
151         temp_cache_line->LRU_next = NULL; //设置缓存行的下一个指针
152         temp_cache_line->LRU_prev = NULL; //设置缓存行的上一个指针
153
154         temp_cache_line->dirty = FALSE; //设置缓存行的脏位
155         insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); //将缓存行插入到LRU列表的头部
156         c1.set_contents[index]++; //更新当前缓存行的有效条目数
157         c1.contents++; //更新缓存中的总的有效条目数
158     }
159     //更新统计信息, 先使用stat设计信息, 这里还未添加all cache的情况
160
161     cache_stat_data.accesses++;
162     cache_stat_data.misses++;
163     cache_stat_data.demand_fetches += words_per_block;
164 }
165 }
166 else {
167     //*****
168     void perform_access(addr, access_type)
169     {
170         unsigned addr, access_type;
171     }
172     /* handle an access to the cache */
173     //开始处理缓存访问
174     int index, tag;
175     //存入的地址为
176     if (cache_split) {
177         //1000:
178     }
179     else {
180         //统一缓存 + You, 3天期 + 完全Unified, variable block size, direct-mapped ca...
181         //计算索引和标签
182         index = (addr & c1.index_mask) >> c1.index_mask_offset;
183         tag = addr >> (c1.index_mask_offset + LOG2(c1.n_sets));
184         //在关联表中查找缓存行
185         //先考虑read, 不考虑write
186         Pcache_line curr_cache_set_head = c1.LRU_head[index]; //当前缓存组
187         if (curr_cache_set_head == NULL) {
188             // 初始状态, 分配一个新的缓存行
189             Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
190             temp_cache_line->tag = tag; //设置缓存行的内容
191             temp_cache_line->LRU_next = NULL; //设置缓存行的下一个指针
192             temp_cache_line->LRU_prev = NULL; //设置缓存行的上一个指针
193
194             if (access_type == TRACE_DATA_STORE) {
195                 temp_cache_line->dirty = TRUE; //设置缓存行的脏位
196             }
197             else {
198                 temp_cache_line->dirty = FALSE; //设置缓存行的脏位
199             }
200             insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); //将缓存行插入到LRU列表的头部
201             c1.set_contents[index]++; //更新当前缓存行的有效条目数
202             c1.contents++; //更新缓存中的总的有效条目数
203         }
204         //更新统计信息
205         if (access_type == TRACE_DATA_LOAD) {
206             //数据命中
207             cache_stat_data.accesses++;
208             cache_stat_data.misses++;
209             cache_stat_data.demand_fetches += words_per_block;
210         }
211         else if (access_type == TRACE_DATA_STORE) {
212             //数据命中
213             cache_stat_data.accesses++;
214             cache_stat_data.misses++;
215             cache_stat_data.demand_fetches += words_per_block;
216         }
217         else if (access_type == TRACE_INST_LOAD) {
218             //指令命中
219             cache_stat_inst.accesses++;
220             cache_stat_inst.misses++;
221             cache_stat_inst.demand_fetches += words_per_block;
222         }
223     }
224     else {
225         //*****
226         void perform_access(addr, access_type)
227         {
228             unsigned addr, access_type;
229         }
230         /* handle an access to the cache */
231         //开始处理缓存访问
232         int index, tag;
233         //存入的地址为
234         if (cache_split) {
235             //1000:
236         }
237         else {
238             //统一缓存 + You, 3天期 + 完全Unified, variable block size, direct-mapped ca...
239             //计算索引和标签
240             index = (addr & c1.index_mask) >> c1.index_mask_offset;
241             tag = addr >> (c1.index_mask_offset + LOG2(c1.n_sets));
242             //在关联表中查找缓存行
243             //先考虑read, 不考虑write
244             Pcache_line curr_cache_set_head = c1.LRU_head[index]; //当前缓存组
245             if (curr_cache_set_head == NULL) {
246                 // 初始状态, 分配一个新的缓存行
247                 Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
248                 temp_cache_line->tag = tag; //设置缓存行的内容
249                 temp_cache_line->LRU_next = NULL; //设置缓存行的下一个指针
250                 temp_cache_line->LRU_prev = NULL; //设置缓存行的上一个指针
251
252                 if (access_type == TRACE_DATA_STORE) {
253                     temp_cache_line->dirty = TRUE; //设置缓存行的脏位
254                 }
255                 else {
256                     temp_cache_line->dirty = FALSE; //设置缓存行的脏位
257                 }
258                 insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); //将缓存行插入到LRU列表的头部
259                 c1.set_contents[index]++; //更新当前缓存行的有效条目数
260                 c1.contents++; //更新缓存中的总的有效条目数
261             }
262             //更新统计信息
263             if (access_type == TRACE_DATA_LOAD) {
264                 //数据命中
265                 cache_stat_data.accesses++;
266                 cache_stat_data.misses++;
267                 cache_stat_data.demand_fetches += words_per_block;
268             }
269             else if (access_type == TRACE_DATA_STORE) {
270                 //数据命中
271                 cache_stat_data.accesses++;
272                 cache_stat_data.misses++;
273                 cache_stat_data.demand_fetches += words_per_block;
274             }
275             else if (access_type == TRACE_INST_LOAD) {
276                 //指令命中
277                 cache_stat_inst.accesses++;
278                 cache_stat_inst.misses++;
279                 cache_stat_inst.demand_fetches += words_per_block;
280             }
281         }
282     }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

图 5: 增加数据写和指令读的统计

如下图 6, 在 cache 命中后, 增加了数据写和指令读的统计信息。

```

183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }
207 }
208 }
209 }
210 }
211 }
212 }
213 }
214 }
215 }
216 }
217 }
218 }
219 }
220 }
221 }
222 }
223 }
224 }
225 }
226 }
227 }
228 }
229 }
230 }
231 }
232 }
233 }
234 }
235 }
236 }
237 }
238 }
239 }
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

图 6: 增加数据写和指令读的统计

如下图 7, 在 cache 未命中后, 增加了数据写和指令读的统计信息。并且增加了对 dirty 标志位的判断, 如果 dirty 标志位为 1, 说明需要将 cache 块的数据写回到内存中, 更新 write back 和 replace 的统计信息。同时在替换 cache 块的时候, 需要根据访问类型, 选择是否设置 dirty 标志位。

```

82     else {
83         // 命中
84
85         cache_stat_data.accesses++;
86         cache_stat_data.demand_fetches += words_per_block;
87         cache_stat_data.replacements++;
88
89         // 命中
90         Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
91         temp_cache_line->tag = tag; // 设置缓存行tag
92         temp_cache_line->LRU_next = NULL; // 设置缓存行的下一个指针
93         temp_cache_line->LRU_prev = NULL; // 设置缓存行的上一个指针
94
95         // 命中
96         temp_cache_line->dirty = FALSE; // 设置缓存行的脏位
97
98         if (c1.set_contents[index] == c1.associativity){
99             // 命中
100             // 命中
101             delete &c1.LRU_head[index], &c1.LRU_tail[index]; // 删除当前缓存行的头指针
102             insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); // 将缓存行插入到LRU链表的头部
103         }
104         else {
105             // 缓存行未命中，需要插入到LRU链表的尾部
106             insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); // 将缓存行插入到LRU链表的尾部
107             c1.set_contents[index]++; // 命中次数增加
108         }
109     }
110 }
111
112 // 命中
113 if (access_type == TRACE_DATA_LOAD) {
114     // 命中
115     cache_stat_data.accesses++;
116     cache_stat_data.demand_fetches += words_per_block;
117     cache_stat_data.replacements++;
118     if (c1.LRU_tail[index]->dirty == TRUE){
119         cache_stat_data.copies_back += words_per_block;
120     }
121 }
122 else if (access_type == TRACE_DATA_STORE) {
123     // 命中
124     cache_stat_data.accesses++;
125     cache_stat_data.demand_fetches += words_per_block;
126     cache_stat_data.replacements++;
127     if (c1.LRU_tail[index]->dirty == TRUE){
128         cache_stat_data.copies_back += words_per_block;
129     }
130 }
131 else if (access_type == TRACE_INST_LOAD) {
132     // 命中
133     cache_stat_inst.accesses++;
134     cache_stat_inst.demand_fetches += words_per_block;
135     cache_stat_inst.replacements++;
136     if (c1.LRU_tail[index]->dirty == TRUE){
137         cache_stat_inst.copies_back += words_per_block;
138     }
139 }
140 }
141 // 命中
142 Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
143 temp_cache_line->tag = tag; // 设置缓存行tag
144 temp_cache_line->LRU_next = NULL; // 设置缓存行的下一个指针
145 temp_cache_line->LRU_prev = NULL; // 设置缓存行的上一个指针
146
147 // 命中
148 if (access_type == TRACE_DATA_STORE){
149     temp_cache_line->dirty = TRUE;
150 }
151 else {
152     temp_cache_line->dirty = FALSE;
153 }
154
155 if (c1.set_contents[index] == c1.associativity){
156     // 命中
157     // 命中
158     Pcache_line curr_cache_set_tail = c1.LRU_tail[index];
159     delete &c1.LRU_head[index], &c1.LRU_tail[index], curr_cache_set_tail; // 删除当前缓存行的头指针
160     insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); // 将缓存行插入到LRU链表的头部
161     free(curr_cache_set_tail);
162 }
163 else {
164     // 缓存行未命中，需要插入到LRU链表的尾部
165     insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line); // 将缓存行插入到LRU链表的尾部
166     c1.set_contents[index]++; // 命中次数增加
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }
207 }
208 }
209 }
210 }
211 }
212 }
213 }
214 }
215 }
216 }
217 }
218 }
219 }
220 }
221 }
222 }
223 }
224 }
225 }
226 }
227 }
228 }
229 }
230 }
231 }
232 }
233 }
234 }
235 }
236 }
237 }
238 }
239 }
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

图 7: 增加数据写和指令读的统计

增加 flush 功能

在增加写入和读取的统计信息后，需要增加 flush 功能，即在 cache 模拟结束后，将 cache 清空，同时将 dirty 标志位为 1 的 cache 块写回到内存中。并更新统计信息。

如下图 8 所示：flush 函数只需要遍历 cache，将 dirty 标志位为 1 的 cache 块写回到内存中，同时更新统计信息。然后释放分配的内存。

```

214 void flush()
215 {
216     /* flush the cache */
217
218 }
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

图 8: 增加 flush 功能

BUG 修复

经过测试，发现 replace 和 write back 的统计信息有问题。经过调试，发现了没有更新 LRU 链表和统计信息时候没有判断 LRU 链表是否已满的 bug

如下图 9 所示：在每次访问后，需要更新 LRU 链表，将访问的 cache 块放在链表的头部。在统计信息时，需要判断 LRU 链表是否已满，只要已满，才要进行替换，增加 replace 和 writeback 的统计信息。

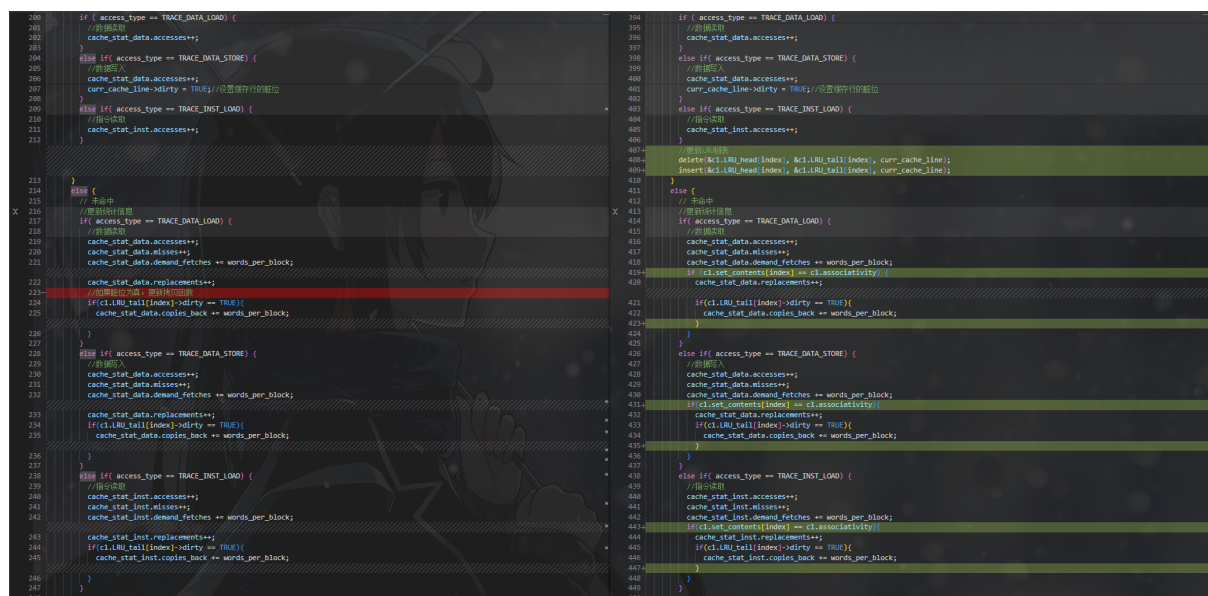


图 9: BUG 修复

3.5 增加 split cache 功能

之后增加 split cache 功能，即将 i-d cache 分开，分别对指令和数据分别进行 cache。

这里的实现思路和 unify cache 的实现思路基本一致，只是需要根据访问的类型选择对应的 cache 进行访问即可，其余的逻辑基本一致。实现的思路和前面的 unify cache 的实现思路基本一致，这里不再赘述。详细过程参考源码。

3.6 增加 write through 和 write no allocate 策略

增加 write through

之后增加 write through 的功能，即写直达策略。写直达策略和写回策略的主要区别在于，写直达策略在写操作时，直接将数据写回到内存中，而写回策略在写操作时，只是将数据写回到 cache 中，只有在 cache 块被替换时，才会将数据写回到内存中。

如下图 10 所示：其实现非常简单，只要在之前代码的基础上，增加对写直达策略的判断即可：

在分配第一个 cache 块的时候, 如果为数据写类型访问且是 write through 策略, 不需要再设置 dirty 标志位, 同时直接增加 write back 的统计信息。

在 cache 命中后, 如果为数据写类型访问且 write through 策略, 需要清除 dirty 标志位设置为 0, 同时增加 write back 的统计信息。

在 cache 未命中后, 如果为数据写类型访问且 write through 策略, 直接增加 write back 的统计信息, 且分配的 cache 块也不需要设置 dirty 标志位。

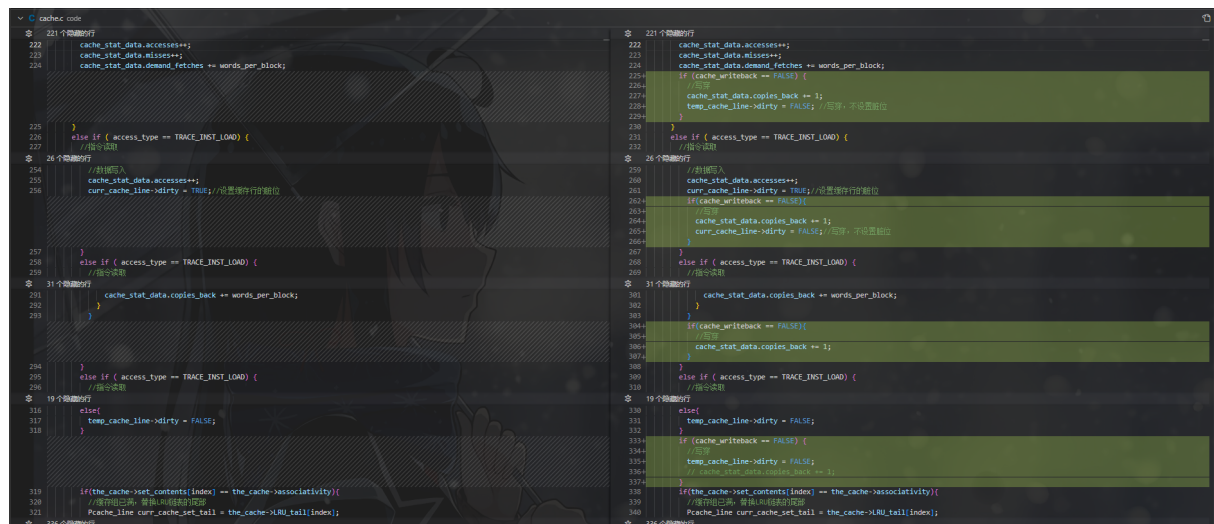


图 10: 增加 write through

增加 write no allocate

最后增加 write no allocate 的功能, 即写不分配策略。写不分配策略和写分配策略的主要区别在于, 写不分配策略在写操作时, 如果 cache 未命中, 则直接将数据写回到内存中, 而不分配 cache 块。而写分配策略在写操作时, 如果 cache 未命中, 则分配一个 cache 块, 并将数据写回到 cache 块中。

如下图 11 所示: 首先在初始分配 cache 块的时候, 如果为写数据写类型访问且是 write no allocate 策略, 则直接跳过分配 cache 块的过程, 直接将数据写回到内存中, 同时增加 write back 的统计信息。

接着在统计信息时, 如果为数据写类型访问且 write no allocate 策略, 直接增加 write back, 同时不需要 demand fetch 的统计信息。

后续在 cache 未命中时, 也是类似的操作, 直接将数据写回到内存中, 增加 write back, 同时不需要 demand fetch, replace 等的统计信息。

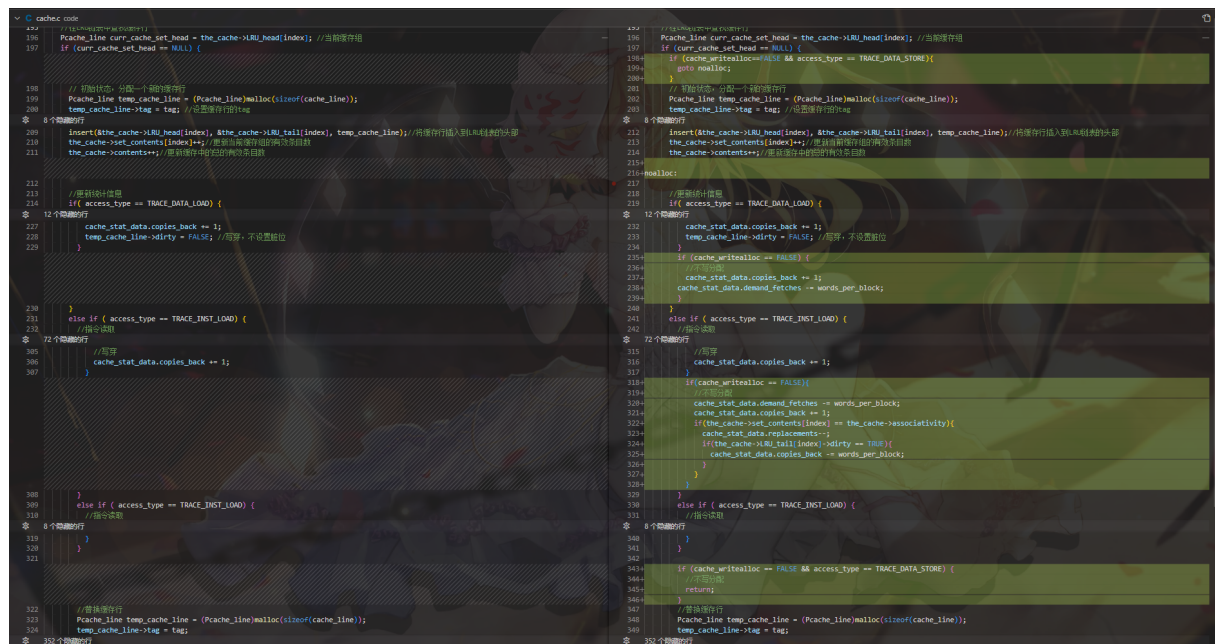


图 11: 增加 write no allocate

测试结果: 写了一个测试脚本进行测试, 测试结果如下: 对比表中的实际值, 成功通过测试。说明 cache 模拟器功能正确实现。


```

valid_spice.sh
...
1  #!/bin/bash
2
3  # 定义要运行的命令列表
4  commands=(
5      "./code/sim.exe -bs 16 -is 8192 -ds 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
6      "./code/sim.exe -bs 16 -is 16384 -ds 16384 -a 1 -wb -wa ./ext_traces/spice.trace"
7      "./code/sim.exe -bs 16 -is 32768 -ds 32768 -a 1 -wb -wa ./ext_traces/spice.trace"
8      "./code/sim.exe -bs 16 -is 65536 -ds 65536 -a 1 -wb -wa ./ext_traces/spice.trace"
9      "./code/sim.exe -bs 16 -us 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
10     "./code/sim.exe -bs 32 -us 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
11     "./code/sim.exe -bs 64 -us 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
12     "./code/sim.exe -bs 128 -is 32768 -ds 32768 -a 1 -wb -wa ./ext_traces/spice.trace"
13     "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 2 -wb -wa ./ext_traces/spice.trace"
14     "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 4 -wb -wa ./ext_traces/spice.trace"
15     "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 8 -wb -wa ./ext_traces/spice.trace"
16     "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 16 -wb -wa ./ext_traces/spice.trace"
17     "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 128 -wb -wa ./ext_traces/spice.trace"
18     "./code/sim.exe -bs 64 -is 1024 -ds 1024 -a 2 -wb -wa ./ext_traces/spice.trace"
19     "./code/sim.exe -bs 64 -is 1024 -ds 1024 -a 8 -wb -wa ./ext_traces/spice.trace"
20     "./code/sim.exe -bs 64 -is 1024 -ds 1024 -a 16 -wb -wa ./ext_traces/spice.trace"
21
22     "./code/sim.exe -bs 16 -is 8192 -ds 8192 -a 1 -wt -wa ./ext_traces/spice.trace"
23     "./code/sim.exe -bs 32 -is 8192 -ds 8192 -a 1 -wt -wa ./ext_traces/spice.trace"
24     "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 2 -wt -wa ./ext_traces/spice.trace"
25
26     "./code/sim.exe -bs 16 -is 8192 -ds 8192 -a 1 -wa -nw ./ext_traces/spice.trace"
27     "./code/sim.exe -bs 32 -is 8192 -ds 8192 -a 1 -wa -nw ./ext_traces/spice.trace"
28     "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 2 -wa -nw ./ext_traces/spice.trace"
29 )
30
31 # 运行命令并提取数据
32 for cmd in "${commands[@]"; do
33     output=$(($cmd))
34
35     # 提取所需的数据
36     misses_instr=$(echo "$output" | grep -A 5 "INSTRUCTIONS" | grep "misses" | awk '{print $2}')
37     replace_instr=$(echo "$output" | grep -A 5 "INSTRUCTIONS" | grep "replace" | awk '{print $2}')
38     misses_data=$(echo "$output" | grep -A 5 "DATA" | grep "misses" | awk '{print $2}')
39     replace_data=$(echo "$output" | grep -A 5 "DATA" | grep "replace" | awk '{print $2}')
40     demand_fetch=$(echo "$output" | grep "demand fetch" | awk '{print $3}')
41     copies_back=$(echo "$output" | grep "copies back" | awk '{print $3}')
42
43     # 打印提取的数据
44     echo -e "$misses_instr\t$replace_instr\t$misses_data\t$replace_data\t$demand_fetch\t$copies_back"
45 done

```

图 12: 测试脚本

```
Huang Xianmin@LAPTOP-POST5089 MINGW64 /d/DesktopLink/jizu/pro2/project2/attachment (rewrite_wt_wna)
• $ ./valid_spice.sh
24681 24173 8283 7818 131856 12024
11514 10560 5839 5051 69412 8008
5922 4321 1567 520 29956 3628
2619 484 1290 103 15636 3324
36136 35787 21261 21098 229588 37844
26673 26502 19561 19476 369872 69104
23104 23029 20377 20324 695696 136112
1964 1726 459 280 77536 7296
6590 6462 3160 3032 156000 18880
6025 5897 875 747 110400 7296
6435 6307 803 675 115808 6656
6536 6408 799 671 117360 6624
6523 6395 790 662 117008 6576
44962 44946 24767 24751 1115664 149200
45885 45869 22808 22792 1099088 112480
45969 45953 20667 20651 1066176 90416
24681 24173 8283 7818 131856 66538
15868 15612 7504 7264 186976 66538
6590 6462 3160 3032 156000 66538
24681 24173 14904 6688 127304 14643
15868 15612 15098 6421 180200 22033
6590 6462 8638 2726 151104 13624
```

图 13: 测试结果

CS	I- vs D-	BS	Ass	Write	Alloc	Instructions		Data		Total	
						Misses	Repl	Misses	Repl	DF	CB
8 K	Split	16	1	WB	WA	24681	24173	8283	7818	131856	12024
16 K	Split	16	1	WB	WA	11514	10560	5839	5051	69412	8008
32 K	Split	16	1	WB	WA	5922	4321	1567	520	29956	3628
64 K	Split	16	1	WB	WA	2619	484	1290	103	15636	3324
8 K	Unified	16	1	WB	WA	36136	35787	21261	21098	229588	37844
8 K	Unified	32	1	WB	WA	26673	26502	19561	19476	369872	69104
8 K	Unified	64	1	WB	WA	23104	23029	20377	20324	695696	136112
32 K	Split	128	1	WB	WA	1964	1726	459	280	77536	7296
8 K	Split	64	2	WB	WA	6590	6462	3160	3032	156000	18880
8 K	Split	64	4	WB	WA	6025	5897	875	747	110400	7296
8 K	Split	64	8	WB	WA	6435	6307	803	675	115808	6656
8 K	Split	64	16	WB	WA	6536	6408	799	671	117360	6624
8 K	Split	64	128	WB	WA	6523	6395	790	662	117008	6576
1 K	Split	64	2	WB	WA	44962	44946	24767	24751	1115664	149200
1 K	Split	64	8	WB	WA	45885	45869	22808	22792	1099088	112480
1 K	Split	64	16	WB	WA	45969	45953	20667	20651	1066176	90416
8 K	Split	16	1	WT	WA	24681	24173	8283	7818	131856	66538
8 K	Split	32	1	WT	WA	15868	15612	7504	7264	186976	66538
8 K	Split	64	2	WT	WA	6590	6462	3160	3032	156000	66538
8 K	Split	16	1	WB	WNA	24681	24173	14904	6688	127304	14643
8 K	Split	32	1	WB	WNA	15868	15612	15098	6421	180200	22033
8 K	Split	64	2	WB	WNA	6590	6462	8638	2726	151104	13624

图 14: 实际值

4 任务 2: 评估不同的高速缓存体系结构特性的性能

Working Set Characterization

任务要求:

使用缓存模拟器，将缓存的命中率绘制为缓存大小的函数。从 4 个字节的缓存大小开始，然后增加大小（每次增加 2 倍），直到命中率对缓存大小保持不敏感。使用拆分缓存组织，以便您可以分别描述指令和数据引用的行为（即，每个示例跟踪将有两个图——一个用于指令，一个用于数据）。通过 * 总是”使用完全关联的缓存来排除冲突遗漏的影响。此外，请始终将块大小设置为 4 个字节，使用回写缓存，并使用写分配策略。

写了一个 python 脚本，用于自动测试不同的 cache 大小，得到不同 cache 大小下的命中率。详情见源码。

最终得到的结果如下图所示:

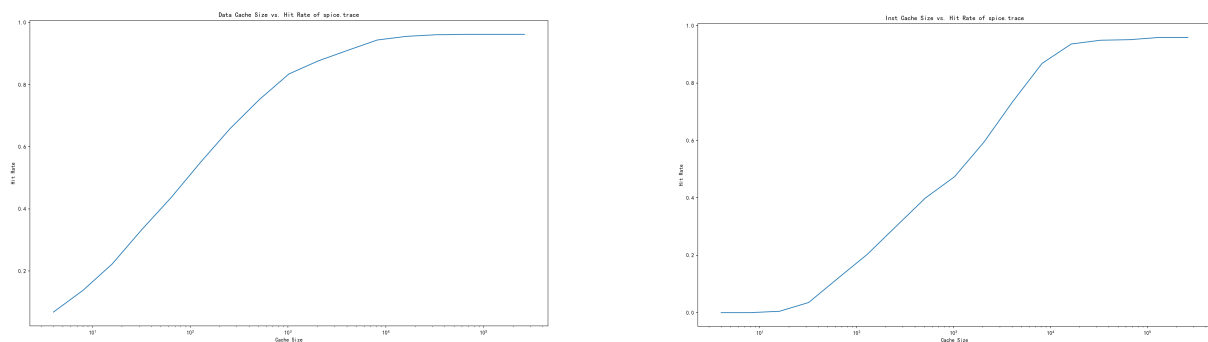


图 15: spice.trace 测试结果

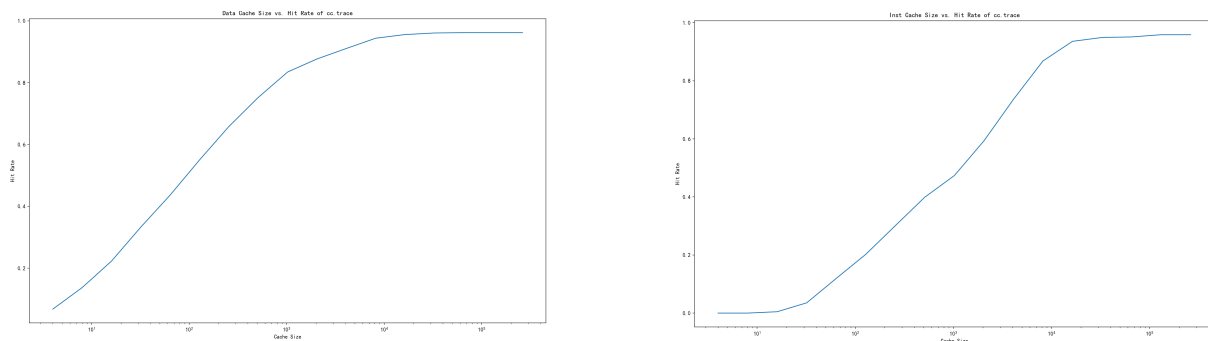


图 16: cc.trace 测试结果

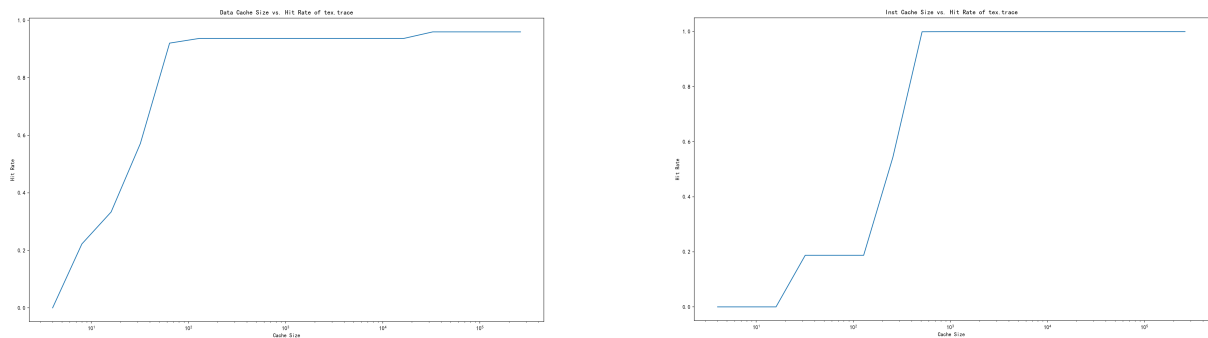


图 17: tex.trace 测试结果

回答问题:

1. 解释一下这个实验在做什么，以及它是如何工作的。此外，请解释命中率与缓存大小图中的特性的特性的重要性。
2. 这三个样本跟踪的总指令工作集大小和数据工作集大小是多少？

问题 1 回答: 实验目的和工作原理 本实验的目的是通过模拟不同缓存大小对命中率的影响，来评估缓存的工作集大小。工作集是指在一定时间内被频繁访问的内存块集合。通过增加缓存大小，我们可以观察到命中率的变化，从而推断出工作集的大小。当缓存大小增加到一定程度，命中率不再显著提高时，表明当前的工作集已经可以被完全包含在缓存中，进一步增加缓存大小不会带来性能上的提升。

实验的工作原理是使用缓存模拟器模拟不同大小的缓存，并记录每次访问时的命中和未命中情况。从 4 字节的缓存大小开始，每次翻倍增加，直到命中率的变化不再显著。通过这种方式，我们可以确定在不同工作负载下，缓存的最佳大小。

命中率与缓存大小图表的意义

- **初始快速上升:** 在缓存大小较小时，命中率随着缓存大小的增加而迅速上升，这表明工作集大小较小，增加缓存可以显著提高命中率。
- **逐渐平缓:** 随着缓存大小的继续增加，命中率的提升速度逐渐减慢，这表明工作集大小正在接近当前缓存大小。
- **趋于平稳:** 当缓存大小增加到一定程度后，命中率趋于平稳，这表明工作集已经被完全包含在缓存中，进一步增加缓存大小不会提高命中率。

问题 2 回答: 根据上面的分析可知：当缓存大小增加到一定程度后，命中率趋于平稳，这表明工作集已经被完全包含在缓存中，进一步增加缓存大小不会提高命中率。因此，当命中率趋于平稳时，工作集的大小即为当前缓存大小。因此，我们可以通过观察命中率与缓存大小图表的特性来推断工作集的大小。

从上面的三张图片可以看出，spice.trace 的数据工作集大小约为 $2^{14} \text{Bytes} = 16 \text{KB}$ ，指令工作集大小也约为 $2^{14} \text{Bytes} = 16 \text{KB}$

cc.trace 的数据工作集大小约为 $2^{14} = 3\text{Bytes} = 16 == 8KB$ ，指令工作集大小约为 $2^{14}\text{Bytes} = 16KB$

tex.trace 的数据工作集大小约为 $2^6\text{Bytes} = 128B$ ，指令工作集大小约为 $2^9\text{Bytes} = 512B$

Impact of Block Size

任务要求:

将缓存模拟器设置为拆分的 i 缓存组织，每个大小为 8k 字节。将关联性设置为 2，使用回写缓存，并使用写分配策略。将高速缓存的命中率绘制为块大小的函数。在 4 字节和 4k 字节之间改变块大小，幂为 2。对这三个轨迹执行此操作，并为指令和数据引用创建一个单独的绘图。

使用 python 脚本，自动测试不同的 block size，得到不同 block size 下的命中率并绘制图表。详情见源码。最终得到的结果如下图所示：

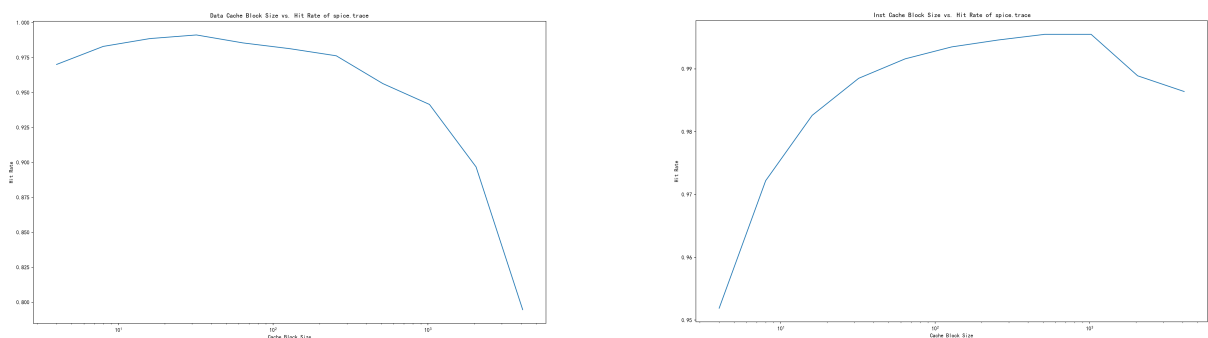


图 18: spice.trace 测试结果

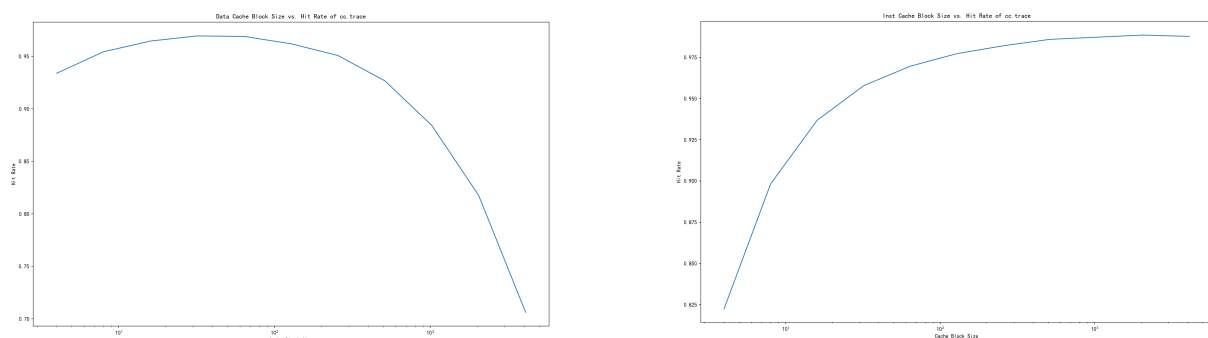


图 19: cc.trace 测试结果

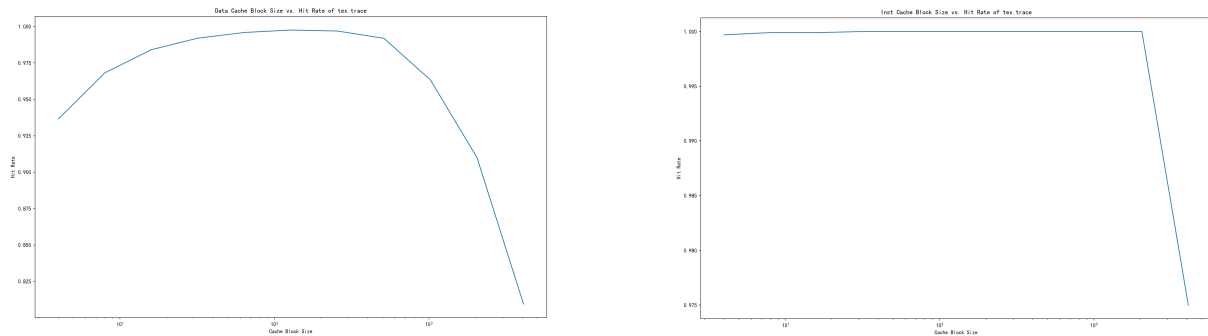


图 20: tex.trace 测试结果

回答问题:

1. 解释命中率与块大小图的形状。特别是，解释空间局部性与这条曲线的形状的相关性。
2. 每个跟踪的最佳块大小是多少（分别考虑指令和数据引用）？
3. 指令和数据引用的最佳块大小是否不同？这告诉你关于指令引用和数据引用的性质是什么？

问题 1 回答:

- 初始上升阶段：当缓存块大小从较小的值开始增加时，命中率迅速上升。这是因为较小的块大小能够更精细地映射内存区域，使得缓存能够更好地捕捉到程序的空间局部性。如果程序访问了一块内存区域，那么相邻的内存区域很可能也会被访问，小的块大小使得这些相邻区域更有可能被一起加载到缓存中。
- 平台期：随着块大小的进一步增加，命中率的增长速度开始放缓，进入一个平台期。在这个阶段，缓存块大小已经足够大，能够覆盖程序的大部分空间局部性需求。即使块大小继续增加，对命中率的提升也不再显著，因为缓存已经能够有效地利用空间局部性。
- 下降阶段：当块大小增加到一定程度后，命中率开始下降。这是因为过大的块大小可能会导致缓存中存储的块包含许多不相关的内存区域，这些区域可能不会被程序访问，从而导致缓存空间的浪费。此外，大的块大小可能会覆盖多个可能被独立访问的内存区域，增加了缓存冲突的可能性，降低了缓存的效率。
- 曲线的拐点：曲线的拐点通常出现在块大小足以覆盖程序的典型空间局部性范围时。在这个点之前，增加块大小可以提高命中率，因为更大的块可以包含更多的局部性区域。在这个点之后，增加块大小反而可能导致命中率下降，因为块内包含的不相关内存区域增多，减少了缓存的有效性。

分析六张图，他们都大致呈现了上述的特性，但是曲线的形状和拐点位置不同，说明不同程序的空间局部性不同，对块大小的需求也不同，所以导致了不同的曲线形状。

问题 2 回答：通过分析每个轨迹的命中率与块大小的关系图，我们可以确定每个轨迹的最佳块大小，即在该块大小下，缓存命中率达到最高或接近最高。

4.1 spice.trace

- **指令引用：**从图中可以看出，当块大小达到 1024 字节时，命中率接近最高，之后随着块大小的增加，命中率略有下降。
- **数据引用：**数据引用的命中率在块大小为 32 字节时达到最高，之后随着块大小的增加，命中率开始下降。

4.2 cc.trace

- **指令引用：**对于指令引用，块大小为 2048 字节时，命中率最高
- **数据引用：**数据引用的命中率在块大小为 32 字节时达到最高，之后随着块大小的增加，命中率略有下降。

4.3 tex.trace

- **指令引用：**在块大小为 4 字节到 2048 字节之间，命中率基本保持稳定，没有明显的变化，但在块大小为 4096 字节时，命中率开始下降。
- **数据引用：**数据引用的命中率在块大小为 128 字节时达到最高，之后随着块大小的增加，命中率开始下降。

问题 3 回答：

通过分析每个轨迹的命中率与块大小的关系图，我们可以确定指令引用和数据引用的最佳块大小。结果显示，指令引用和数据引用的最佳块大小确实不同。

这些结果表明，指令引用和数据引用的性质存在显著差异。指令引用通常具有较强的空间局部性，因为程序的指令通常是顺序执行的，因此较大的块大小可以更好地利用这种局部性。而数据引用则可能更加分散，具有较弱的空间局部性，因此较小的块大小更适合数据引用，以减少缓存空间的浪费和缓存冲突的可能性。

Impact of Associativity

任务要求:

将缓存模拟器设置为拆分的缓存组织，每个大小为 8k 字节。将块大小设置为 128 字节，使用回写缓存，并使用写分配策略。绘制高速缓存的命中率作为关联性的函数。改变 1 和 64 之间的结合性，幂次为 2。对这三个轨迹执行此操作，并为指令和数据引用创建一个单独的绘图。

使用 python 脚本, 自动测试不同的 associativity, 得到不同 associativity 下的命中率并绘制图表。详情见源码。

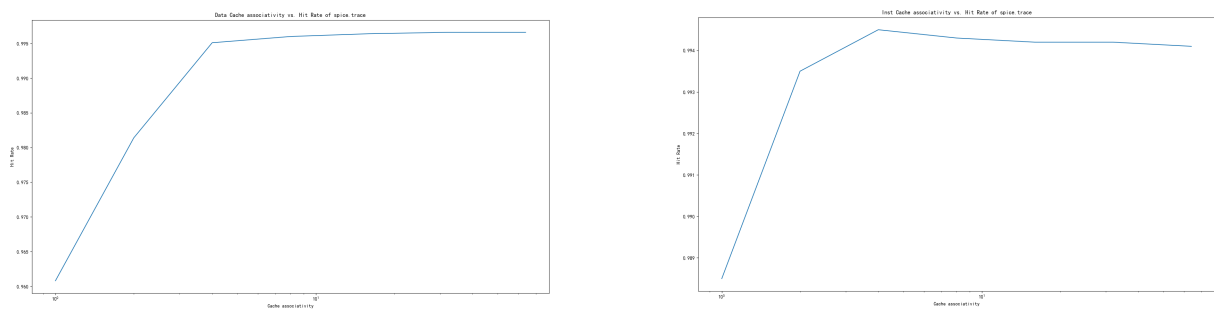


图 21: spice.trace 测试结果

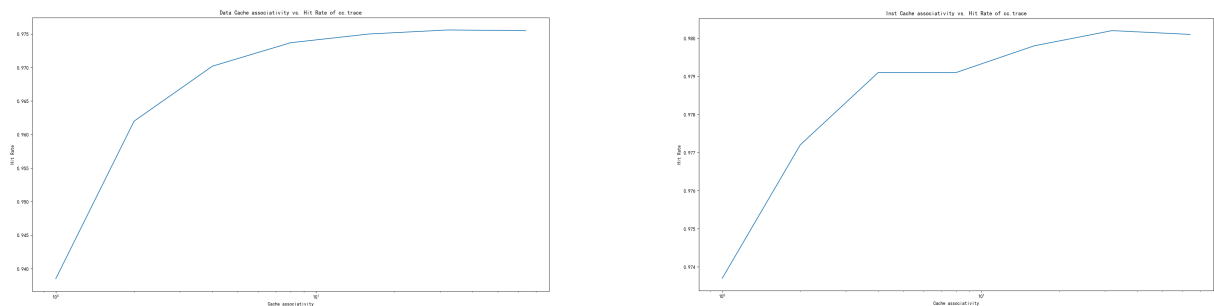


图 22: cc.trace 测试结果

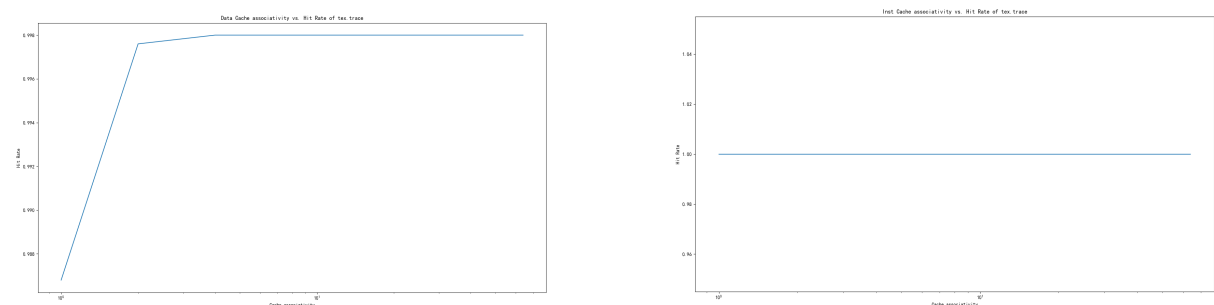


图 23: tex.trace 测试结果

回答问题:

1. 解释为什么 hit rate vs. associativity 的图表具有这样的形状。
2. 指令和数据的图之间是否有区别？这告诉你什么，关于 associativity 对指令和数据引用的影响的差异？

问题 1 回答:

命中率与关联度关系图的形状解释

命中率与关联度的关系图展示了缓存命中率随关联度变化的趋势。这种趋势的形状与缓存的组织结构和程序的内存访问模式密切相关。

初始上升阶段

subsubsection* 在关联度较低时，命中率随着关联度的增加而迅速上升。这是因为较低的关联度意味着缓存中的块冲突较多，增加关联度可以减少这些冲突，从而提高命中率。

平台期

当关联度增加到一定程度后，命中率的提升速度开始放缓，进入一个平台期。在这个阶段，缓存已经能够较好地减少冲突，进一步增加关联度对命中率的提升效果有限。

某些情况出现下降阶段

在某些情况下，当关联度继续增加超过某个阈值后，命中率可能会略有下降。这可能是由于缓存结构的复杂性增加，导致缓存管理的开销增大，或者在某些极端情况下，过大的关联度可能导致缓存的利用率降低。

其它特殊情况:

如 tex.trace 的 icache 图，命中率随着关联度几乎保持在 100%，说明该程序的指令引用具有很好的空间局部性，无论关联度如何，都能被缓存很好地覆盖。

结论

缓存的关联度对命中率有显著影响。合理选择关联度对于优化缓存性能至关重要。在实际应用中，需要根据程序的特性和访问模式来调整关联度，以达到最佳的缓存效率。

问题 2 回答:

差异分析

- **上升阶段:** 对于指令和数据引用, 命中率随着关联度的增加而上升, 但上升的速率和开始平稳的点可能不同。
- **平台期:** 在达到一定关联度后, 命中率的提升趋于平缓, 这个点对于指令和数据引用可能有所不同。
- **下降阶段:** 在某些情况下, 过高的关联度可能导致命中率略有下降, 这在指令和数据引用中的表现可能不一致。

差异意义

关联度对指令和数据引用的影响差异反映了两者在缓存行为上的不同:

- **指令引用:** 由于指令访问通常具有较好的空间局部性, 适当的关联度可以显著提高命中率。
- **数据引用:** 数据访问模式可能更加随机, 因此关联度对命中率的影响可能不如指令引用明显。

结论

关联度对指令和数据引用的命中率有不同的影响。指令引用可能更受益于较高的关联度, 而数据引用在达到一定关联度后, 命中率的提升可能有限。这表明在设计缓存时, 需要考虑不同类型内存访问的特性, 以优化缓存性能。

Memory Bandwidth

任务要求 1:

使用缓存模拟器，比较通写缓存和回写缓存生成的总内存流量。使用拆分的高速缓存组织。模拟一些合理的缓存大小（如 8K 字节和 16K 字节）、合理的块大小（如 64 和 128 字节）以及合理的关联性（如 2 和 4）。现在，请使用无写分配策略。总共运行 4 或 5 个不同的模拟。

回答问题:

1. 哪个缓存具有较小的内存流量，通写缓存或回写缓存？为什么
2. 在什么情况下，你对上面的问题的回答会翻转吗？解释。

使用 python 脚本, 自动测试不同的 cache size, block size, associativity, write policy, 得到不同参数下的内存流量并绘制图表。得到如下结果:

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	rw	151104	13624
128	8192	8192	2	wb	rw	286768	32387
64	16384	16384	2	wb	rw	57008	8252
128	16384	16384	2	wb	rw	93632	9880
64	8192	8192	4	wb	rw	107296	9219
128	8192	8192	4	wb	rw	162240	13733
64	16384	16384	4	wb	rw	38064	7681
128	16384	16384	4	wb	rw	64320	7668

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wt	rw	151104	72322
128	8192	8192	2	wt	rw	286768	72777
64	16384	16384	2	wt	rw	57008	71430
128	16384	16384	2	wt	rw	93632	71490
64	8192	8192	4	wt	rw	107296	71453
128	8192	8192	4	wt	rw	162240	71599
64	16384	16384	4	wt	rw	38064	71047
128	16384	16384	4	wt	rw	64320	71086

图 24: spice.trace 测试结果

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	rw	474256	39426
128	8192	8192	2	wb	rw	824192	78672
64	16384	16384	2	wb	rw	252096	24048
128	16384	16384	2	wb	rw	440768	42538
64	8192	8192	4	wb	rw	423104	32102
128	8192	8192	4	wb	rw	716736	59292
64	16384	16384	4	wb	rw	205120	10588
128	16384	16384	4	wb	rw	353632	31103

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wt	rw	474256	94312
128	8192	8192	2	wt	rw	824192	92582
64	16384	16384	2	wt	rw	252096	91382
128	16384	16384	2	wt	rw	440768	90416
64	8192	8192	4	wt	rw	423104	92940
128	8192	8192	4	wt	rw	716736	92370
64	16384	16384	4	wt	rw	205120	90954
128	16384	16384	4	wt	rw	353632	90669

图 25: cc.trace 测试结果

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	rw	2880	29903
128	8192	8192	2	wb	rw	3488	29935
64	16384	16384	2	wb	rw	2464	29909
128	16384	16384	2	wb	rw	2656	29957
64	8192	8192	4	wb	rw	3312	29903
128	8192	8192	4	wb	rw	4064	29935
64	16384	16384	4	wb	rw	2464	29909
128	16384	16384	4	wb	rw	2656	29957

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wt	rw	2880	134384
128	8192	8192	2	wt	rw	3488	134384
64	16384	16384	2	wt	rw	2464	134374
128	16384	16384	2	wt	rw	2656	134374
64	8192	8192	4	wt	rw	3312	134384
128	8192	8192	4	wt	rw	4064	134384
64	16384	16384	4	wt	rw	2464	134374
128	16384	16384	4	wt	rw	2656	134374

图 26: tex.trace 测试结果

问题 1 回答: 对比图中的 `demand_fetch` 和 `copies_back` 可以发现 `write back cache` 具有较小的内存流量。这是因为回写缓存策略下，只有当数据被修改时，才会将数据写回主存，而通写缓存策略下，每次数据修改都会立即写回主存，导致更多的写操作，从而增加内存流量。

问题 2 回答: 在以下情况下，通写缓存的内存流量可能会比回写缓存小：

- **写操作频繁且数据不经常被重写：** 如果程序中写操作非常频繁，并且写入的数据很少被重写，那么通写缓存可能会比回写缓存更有效。因为在回写缓存中，每次写操作都需要将数据标记为“脏”，并在替换时写回主存，这会导致额外的内存流量。而在通写缓存中，每次写操作都会立即写回主存，避免了多次写回相同数据的情况。
- **数据一致性要求高：** 在某些系统中，数据一致性要求很高，需要确保每次写操作后数据立即更新到主存。在这种情况下，通写缓存可以减少由于数据一致性问题而导致的额外内存流量。
- **缓存替换频繁：** 如果缓存替换非常频繁，回写缓存可能会导致大量的写回操作，从而增加内存流量。而通写缓存每次写操作都立即写回主存，避免了替换时的写回操作。

总的来说，当写操作频繁且数据不经常被重写，或者系统对数据一致性要求高时，通写缓存的内存流量可能会比回写缓存小。

任务要求 2:

现在使用相同的参数，但修复策略以进行回写。执行相同的实验，但这次比较写分配和写不分配策略。

回答问题:

1. 哪个缓存具有较小的内存流量、写分配缓存或无写分配缓存？为什么
2. 在什么情况下，你对上面的问题的回答会翻转吗？解释。

使用 python 脚本, 自动测试不同的 `cache size`, `block size`, `associativity`, `write policy`, 得到不同参数下的内存流量并绘制图表。得到如下结果:

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	na	156000	18880
128	8192	8192	2	wb	na	291584	36256
64	16384	16384	2	wb	na	59856	6208
128	16384	16384	2	wb	na	99008	9680
64	8192	8192	4	wb	na	110400	7296
128	8192	8192	4	wb	na	171136	14592
64	16384	16384	4	wb	na	39728	5280
128	16384	16384	4	wb	na	66304	7200

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	rw	151104	13624
128	8192	8192	2	wb	rw	286768	32287
64	16384	16384	2	wb	rw	57008	8252
128	16384	16384	2	wb	rw	95632	9880
64	8192	8192	4	wb	rw	107296	9219
128	8192	8192	4	wb	rw	162240	13733
64	16384	16384	4	wb	rw	38064	7681
128	16384	16384	4	wb	rw	64320	9668

图 27: spice.trace 测试结果

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	na	488976	41840
128	8192	8192	2	wb	na	848832	91744
64	16384	16384	2	wb	na	261248	23584
128	16384	16384	2	wb	na	453840	46048
64	8192	8192	4	wb	na	435408	33216
128	8192	8192	4	wb	na	737632	68384
64	16384	16384	4	wb	na	212176	18208
128	16384	16384	4	wb	na	363392	32512

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	rw	434256	39426
128	8192	8192	2	wb	rw	824192	78872
64	16384	16384	2	wb	rw	252096	24048
128	16384	16384	2	wb	rw	440768	45336
64	8192	8192	4	wb	rw	423104	32102
128	8192	8192	4	wb	rw	716736	59292
64	16384	16384	4	wb	rw	205120	19588
128	16384	16384	4	wb	rw	353632	31103

图 28: cc.trace 测试结果

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	na	15696	7664
128	8192	8192	2	wb	na	18528	8608
64	16384	16384	2	wb	na	11696	7616
128	16384	16384	2	wb	na	13696	8256
64	8192	8192	4	wb	na	15608	7568
128	8192	8192	4	wb	na	15648	7648
64	16384	16384	4	wb	na	10480	7568
128	16384	16384	4	wb	na	11200	7648

bs	is	ds	a	write_policy	allocation_policy	demand_fetch	copies_back
64	8192	8192	2	wb	rw	2880	29903
128	8192	8192	2	wb	rw	3488	29935
64	16384	16384	2	wb	rw	2664	29909
128	16384	16384	2	wb	rw	2656	29957
64	8192	8192	4	wb	rw	3312	29903
128	8192	8192	4	wb	rw	4064	29935
64	16384	16384	4	wb	rw	2664	29909
128	16384	16384	4	wb	rw	2656	29957

图 29: tex.trace 测试结果

问题 1 回答: 观察上面的图表可以发现：对于 spice.trace 和 tex.spice 测试结果，写不分配缓存具有较小的内存流量；对于 tex.trace 测试结果，写分配缓存具有较小的内存流量。通常情况下，写不分配策略可能会产生较小的内存流量。原因在于写不分配策略在写操作发生时不会将新的缓存行加载到缓存中，这可以减少从主存取数据的次数，从而减少内存流量。然而，写分配策略会在写操作时加载新的缓存行，这可能会导致更多的缓存替换和对主存的写操作，从而增加内存流量。但是，这种差异也取决于具体的缓存大小、块大小和程序的访问模式。

问题 2 回答: 在程序的空间局部性较高，写操作频繁且数据经常被重写的情况下，写分配缓存可能会产生较小的内存流量。因为写分配缓存会在写操作时加载新的缓存行，并在之后的写操作中重复使用这些缓存行，从而减少了从主存取数据的次数，减少了内存流量。

5 项目总结

本项目通过实现一个基于追踪的缓存模拟器，深入评估了不同缓存结构特性对性能的影响。在项目过程中，我们不仅加深了对缓存工作原理的理解，而且通过实践提高了解决复杂问题的能力。

主要成果

- 成功实现了一个功能全面的缓存模拟器，支持多种缓存配置，包括缓存大小、块大小、关联度、写策略和分配策略。
- 通过模拟不同的缓存配置，我们评估了工作集大小、块大小、关联度和内存带宽对缓存性能的影响。
- 对比了写直达和写回策略在不同缓存配置下的内存流量，为缓存设计提供了实证数据支持。
- 分析了写分配和写不分配策略在特定条件下的性能差异，进一步优化了缓存策略的选择。

经验与反思

- 项目过程中，我们遇到了一些技术挑战，如模拟器的稳定性和性能优化，通过对代码的不断调试，最终克服了这些挑战。
- 实践中我们发现，缓存设计需要综合考虑程序特性和硬件限制，没有一种配置能够适用于所有场景。
- 通过项目，我们更加认识到理论与实践相结合的重要性，以及持续学习和适应新技术的必要性。